

Lekcja 11: Flappy pełny

Cel

Z prototypu robisz **skończoną grę**: ekran tytułowy, rozgrywka, ekran końca z wynikiem, rekord, czas. Nauczysz się `enum` i `switch` – porządnego sposobu na „w jakim stanie jest gra”.

Co nowego

Dotąd „stan gry” trzymałeś w `bool dead`. Przy trzech ekranach (tytuł, gra, koniec) potrzebujesz zmiennej, która ma **kilka nazwanych wartości**:

```
enum State { TITLE, PLAYING, GAME_OVER }; // trzy mozliwe wartosci
State state = TITLE; // zmienna typu State
state = PLAYING; // zmiana stanu
```

Z takim stanem dobrze współpracuje `switch` – zamiast łańcucha `if / else if`:

```
switch (state) {
    case TITLE: /* rysuj tytuł */ break;
    case PLAYING: /* gra */ break;
    case GAME_OVER: /* wynik */ break;
}
```

`break` na końcu każdej gałęzi jest ważny – bez niego wykonanie „przelewa się” do następnej.

Maszyna stanów to sposób myślenia: rysujesz kółka (stany) i strzałki (co powoduje przejście). TITLE →(A)→ PLAYING →(zderzenie)→ GAME_OVER →(A)→ PLAYING. Każda większa gra jest tak zbudowana – Lake Mario też (Title, Playing, Dying, LevelClear, GameOver w `mario_game.h`).

Uruchom

Otwórz `gra.cpp`, **Ctrl+Shift+B**. Ekran tytułowy, **Z** startuje. Trzy rury naraz (tablica struktur), wynik na środku góry. Po zderzeniu czarna ramka z wynikiem, **Z** gra od nowa. Zauważ: `restart()` nie jest potrzebne – `start_round()` ustawia wszystko sam, a `setup()` tylko przełącza na tytuł.

Jak to działa

`frame()` to jeden `switch`. Każdy stan rysuje co innego i sprawdza inne warunki przejścia. Logika lotu jest w `update_playing()`, rysowanie świata w `draw_world()` – GAME_OVER używa tego samego rysowania, tylko bez ruchu (świat zamiera). Rury w tablicy: gdy jedna wyjedzie za lewą krawędź, wraca za ostatnią (`x + PIPES * PIPE_SPACING`).

Zadania

- ★ Ekran tytułowy z życiem: ptak na tytule „faluje” (`bird.y = 240 + ...` z `frame_count()`), napis „WCISNIJ A” miga (`(frame_count() / 30) % 2 == 0`).
- ★★ **Czas lotu.** `int start_frame`; ustawiany w `start_round()`; w `PLAYING` licz `flight_time = (frame_count() - start_frame) / 60`; i pokaż w rogu. W `GAME_OVER` wypisz „CZAS: N S”. `watch("time", flight_time)`; – zmienna musi być globalna, żeby `GAME_OVER` ją widział. (Nie nazywaj jej `time` – tak nazywa się funkcja z biblioteki C i komputer może się pogubić.) (test 1)
- ★★ **Rekord.** `int best = 0`; – i **nie zeruj go** w `setup()` ani `start_round()`! W `GAME_OVER`: `if (score > best) best = score`; , pokaż „REKORD”. Pamiętasz z lekcji 02, że zmienne globalne żyją między wejściami z menu? Tu to zaleta: rekord trzyma się, dopóki konsola jest włączona.
- ★★ **Trudność rośnie:** `GAP` na zmienną `gap`, co 5 punktów `gap -= 10` (nie mniej niż 100), przekaż `gap` do `make_pipe`.
- ★★★ **Czwarty stan: PAUSED?** Nie – `START` należy do konsoli. Zamiast tego stan `READY`: po A z tytułu ptak wisi nieruchomo z napisem „LEC!”, a rury stoją, aż gracz machnie po raz pierwszy. Dorysuj do tego medale w `GAME_OVER`: brązowy za 5 punktów, srebrny za 10, złoty za 20 (`circle` odpowiednim kolorem).

Sprawdź sam

Po zadaniu 2: start na 5. klatce, trzy machnięcia co 40 klatek (ptak unosi się około 2 sekund), potem spada i rozbija się:

```
console_sim.exe --game flappy_pelny --hold A 5 6 --hold A 30 31 --hold A 70 71 --hold A 110 111
--frames 300 --trace 50
```

W ostatniej linii `state=GAME_OVER` i `time` co najmniej 1. Bez machania ptak ginie po pół sekundy i `time` zostaje 0 – sprawdź, to też ciekawe.

Dla ciekawych

Otwórz `src/games/mario/mario_game.cpp` i znajdź `switch (state_)` w funkcji `update`. To ten sam wzorec, tylko stanów jest pięć. Znajdziesz też `enum class` – bezpieczniejszą odmianę `enum`, gdzie pisze się `State::Title`.

Słowniczek

po polsku	w C++
typ z nazwanymi wartościami	<code>enum Nazwa { A, B, C };</code>
wyбір gałęzi według wartości	<code>switch (x) { case A: ...; break; }</code>
maszyna stanów	zmienna stanu + przejścia w <code>if</code>
zmienna, która przeżywa restart	globalna, nie zerowana w <code>setup()</code>

